ASYNC AND AWAIT IN DART

Async And Await In Dart

Async/await is a feature in Dart that allows us to write asynchronous code that looks and behaves like synchronous code, making it easier to read.

When a function is marked **async**, it signifies that it will carry out some work that could take some time and will return a Future object that wraps the result of that work.

The **await** keyword, on the other hand, allows you to delay the execution of an async function until the awaited Future has finished. This enables us to create code that appears to be synchronous but is actually asynchronous.

The **async** and **await** keywords both provide a declarative way to define an asynchronous function and use their results. You can use the **async** keyword before a function body to make it asynchronous. You can use the **await** keyword to get the completed result of an asynchronous expression.

Important Concept

- To define an Asynchronous function, add async before the function body.
- The await keyword work only in the async function.

Example 1: Synchronous Function

```
void main() {
  print("Start");
  getData();
  print("End");
}

void getData() {
  String data = middleFunction();
  print(data);
}

Future<String> middleFunction(){
  return Future.delayed(Duration(seconds:5), ()=> "Hello");
```

```
}
```

Show Output

Example 2: Asynchronous function

```
void main() {
  print("Start");
  getData();
  print("End");
}

void getData() async{
  String data = await middleFunction();
  print(data);
}

Future<String> middleFunction(){
  return Future.delayed(Duration(seconds:5), ()=> "Hello");
}
```

Show Output

In the above example, `async` handles the states of the program where any part of the program can be executed.`async` always comes with `await` because `await` holds the part of the program until the rest of the program executed.

Handling Errors

You can handle errors in the dart async function by using `try-catch`. You can write try-catch code the same way you write synchronous code.

Example 3: Handling Errors

```
main() {
  print("Start");
  getData();
  print("End");
}


void getData() async{
```

```
    try{
        String data = await middleFunction();
        print(data);
    }catch(err){
        print("Some error $err");
    }

}

Future<String> middleFunction(){
  return Future.delayed(Duration(seconds:5), ()=> "Hello");
}
```

› Show Output

In the above example, `try-catch` handles the exception that could come after the program is executed.

Info

Note: We cannot perform an asynchronous operation from a synchronous function.

Important Terms

- **async** The async keyword can be used before a function's body to indicate that a function is asynchronous.
- **async function** Functions marked with the async keyword are known as async functions.
- **await** The completed output of an asynchronous expression can be retrieved with the await keyword. Only async functions can use the await keyword.